

CloudGuard -- Complete Beginner's Manual

A "start from literal zero" guide to what this project is, every tool it

A complete, from-zero build manual for the CloudGuard Hybrid Cloud Configuration Change Impact Analyzer.

Table of Contents

Part 1: What Is This Project, In Plain English	5
The problem	5
What this tool actually does	5
A simple analogy	5
Part 2: Glossary -- Every Tool and Term Explained	6
Cloud computing basics	6
Infrastructure as Code (IaC)	6
The Python side of this project	7
Git, GitHub, and automation	7
Testing tools (used only to validate the project, not part of the product itself)	8
This project's own vocabulary	8
Part 3: The Method We Used, and Why Not the Alternatives	10
The method, named	10
Alternative Method A (rejected): Live cloud monitoring	10
Alternative Method B (rejected): Manual code review only	10
Alternative Method C (considered, partially rejected): Machine learning / AI prediction	10
Why our chosen method wins for this project's actual goal	10
Part 4: Who / When / Where / Why to Use This Project	12
Part 5: Complete Step-by-Step Setup From Absolute Zero	13
Step 0 -- Check what you're starting with	13
Step 1 -- Install Python	13
Step 2 -- Install Git	13

Step 3 -- Get the project code	13
Step 4 -- Install the Python libraries this project needs	13
Step 5 -- Set up configuration	14
Step 6 -- Create some example history data	14
Step 7 -- Try the command-line version first (simplest, fastest to verify everything works)	14
Step 8 -- Run the full web dashboard	14
Step 9 -- Run the automated tests (confirms nothing is broken)	14
Step 10 -- Push this to your own GitHub, so the automated bot can run	14
Step 11 (optional) -- Validate against a *real* Terraform-generated plan, not just our sample files	15
Step 12 (optional, advanced) -- Connect a real AWS account, read-only only	17

Part 6: Troubleshooting Log -- Every Real Error We Hit, and Why 18

Error #1 -- pip install shows a wall of WARNING: The script ... is installed in '...\Scripts' which is not on PATH. ¹⁸	
Error #2 -- python -m pytest says No module named pytest, even though you just installed it	18
Error #3 -- git push fails with Permission ... denied to (HTTP 403)	18
Error #4 -- The bot's report crashes with UnicodeEncodeError: 'charmap' codec can't encode character '\U0001f6e1'	18
Error #5 -- wsl --install fails with Forbidden (403).	18
Error #6 -- terraform: command not found, right after winget install Hashicorp.Terraform said it succeeded	19
Error #7 -- Can't copy-paste between a VirtualBox virtual machine and the main computer	19
Error #8 -- Docker Desktop requires WSL2, which requires a restart, which requires admin rights	19
Error #9 -- The "confidence score" was quietly measuring the wrong thing	19
Error #10 -- A change inside a Terraform module reported 0 affected resources, even though a real dependent resource existed	20

Part 7: Honest Limitations -- What's Solid, What Isn't 21

Part 8: Cheat Sheet for Explaining This Project Out Loud 22

uses, why it's built the way it is, and exactly how to build it again on a brand-new laptop -- including every real error we hit while building it the first time, and why each one happened.*

This document assumes you know **nothing**. Every term is explained the first time it's used. If you already know a section, skip it -- nothing bad happens if you do.

Part 1: What Is This Project, In Plain English

The problem

Companies run their computer systems ("infrastructure" -- servers, databases, networking rules) on rented cloud computers instead of buying physical machines. Over time, engineers make small changes to this infrastructure: "let's open this port," "let's resize this database," "let's tighten this security rule." Most of the time these changes are fine. Sometimes, a small, well-intentioned change breaks something else that depends on it -- and the person making the change usually cannot see that dependency, because in a real company, dozens of engineers share the same infrastructure and nobody has the full picture in their head.

This is not a rare/theoretical problem. Several famous, real internet outages (a well-known 2017 AWS S3 outage, and multiple large outages at other companies) trace back to exactly this: an authorized, intentional change that cascaded further than the person making it expected.

What this tool actually does

This project reads a description of a **proposed** infrastructure change (before it happens -- more on this format, called a "Terraform plan," in the glossary) and answers three questions automatically:

1. **What else does this touch?** (built by tracing a map of how all your infrastructure pieces connect to each other)
2. **How risky is it?** (based on what kind of resource it is, what's downstream of it, and whether it touches a security-sensitive setting)
3. **What should you do about it?** (a specific recommendation, and the exact steps to undo it if something goes wrong)

It can show this to a person on a webpage (the "dashboard"), print it to a terminal (the "CLI"), or automatically post it as a comment on a GitHub pull request the moment someone proposes a risky change (the "bot") -- and can even **block** that change from being merged if it's dangerous enough.

A simple analogy

Think of it like a **spell-checker, but for infrastructure changes**. A spell-checker doesn't stop you from writing -- it just flags "this word looks wrong" before you hit send. This tool doesn't stop an engineer from changing infrastructure -- it flags "this change looks risky, here's why, here's what it'll affect" before they hit "apply."

Part 2: Glossary -- Every Tool and Term Explained

Read this section like a dictionary. Come back to it any time a word in Part 5 or Part 6 is unfamiliar.

Cloud computing basics

- **The Cloud:** Renting computing power (a virtual computer, a database, storage space) from someone else's data center over the internet, instead of buying and maintaining your own physical computer.
- **AWS (Amazon Web Services):** Amazon's cloud platform -- one of the biggest "cloud landlords." Competitors: **Azure** (Microsoft), **GCP** (Google Cloud Platform).
- **Resource:** Anything you create inside a cloud account. Examples:
 - **EC2 instance:** A virtual computer you rent to run a program/website.
 - **Security Group:** A firewall -- a list of rules saying "only allow this kind of network traffic in/out." The single most common thing this project analyzes, because misconfiguring it is a classic way to accidentally expose something to the entire internet.
 - **RDS instance:** A database AWS manages for you (so you don't have to install and maintain the database software yourself). Stands for "Relational Database Service."
 - **S3 bucket:** A place to store files ("objects") in the cloud.
 - **IAM (Identity and Access Management):** AWS's permission system -- controls who/what is allowed to do what. An "IAM policy" is a document that spells out exactly which actions are allowed.

Infrastructure as Code (IaC)

- **Infrastructure as Code:** Instead of manually clicking buttons in AWS's website to create resources, you write a **text file** describing the infrastructure you want to exist. This file can be saved, shared, and tracked over time, the same way you'd track any other code.
- **Terraform:** The most popular *tool* that reads an Infrastructure-as-Code text file and actually creates/updates the real resources to match it. Works with AWS, Azure, and GCP alike (which is why our project can be "hybrid cloud," not AWS-only).
- **Terraform plan:** Before Terraform changes anything for real, it can show you a **preview**: "here's exactly what I'm about to create/change/delete." This preview, saved as a file, is the actual input our tool reads. It never touches a real cloud account itself -- it just reads this preview file.
- **Terraform module:** A reusable "package" of infrastructure code -- like a function you call instead of copy-pasting the same setup repeatedly. Real companies almost always organize their Terraform code into modules. (This mattered a lot -- see Part 6, the module bug.)
- **terraform init:** The command that downloads the plugin Terraform needs to talk to a specific cloud provider (e.g., the AWS plugin).

- `terraform show -json <planfile>`: Converts Terraform's plan preview into a machine-readable JSON file -- this is the exact file format our tool's parser reads.

The Python side of this project

- **Python**: The programming language the entire tool is written in.
- **Flask**: A small Python framework used to build the web dashboard/API.
- **NetworkX**: A Python library for working with "graphs" (a graph here means a network of connected dots -- not a chart). We use it to represent "which infrastructure resource depends on which other one," and to calculate how far a change's effects can spread.
- **SQLAlchemy**: A Python library that lets our code talk to a database without writing raw database-query language by hand.
- **MySQL / SQLite**: Two different database systems that store data permanently on disk. MySQL is a full, separate database server (what the original project plan specified); SQLite is a single, simple file requiring zero setup (what this project defaults to, for convenience -- you can switch to MySQL any time by changing one setting).
- **Boto3**: The official Python library for talking to AWS programmatically (instead of clicking around AWS's website).
- **pytest**: A tool that runs automated tests -- small programs that check our code behaves correctly, so we can catch mistakes before a human ever notices them.

Git, GitHub, and automation

- **Git**: A tool that tracks every change ever made to a set of files over time, so you can see history, undo mistakes, and work with others without overwriting each other's work.
- **GitHub**: A website that hosts Git projects online, so multiple people can collaborate on the same code.
- **Repository ("repo")**: A single project's entire folder + its full history, as tracked by Git.
- **Commit**: A saved snapshot of changes, with a message describing what changed and why.
- **Branch**: A parallel, separate line of work, so you can make changes without affecting the main/official version until you're ready.
- **Push**: Uploading your local commits to GitHub.
- **Fork**: Making your own personal copy of someone else's GitHub repository, under your own account.
- **Pull Request (PR)**: A request to merge one branch's changes into

another (usually into the main branch), which other people can review, comment on, and approve before it happens.

- **CI/CD (Continuous Integration / Continuous Deployment):** The general idea of automatically running checks (tests, scans, custom tools) every time someone proposes a code change, instead of relying on a human to remember to check manually.

- **GitHub Actions:** GitHub's built-in system for running automated scripts ("workflows") in response to events, like "someone opened a pull request." This is what runs our "bot."

- **"Bot":** In this context, just an automated script that comments on GitHub -- the same category of thing as GitHub's own "Dependabot." **Not artificial intelligence** -- it's a fixed set of rules, not a learning/predicting model. Worth remembering: this distinction matters if anyone asks "is this AI?" in a viva -- the honest answer is no, and that's a deliberate, defensible design choice (see Part 3).

Testing tools (used only to validate the project, not part of the product itself)

- **Docker:** A tool for packaging and running software in an isolated "container," so it behaves the same everywhere. Many tools (like LocalStack) require Docker to run.

- **WSL (Windows Subsystem for Linux):** A Windows feature that lets you run a real Linux environment alongside Windows. Docker Desktop on Windows requires this as its backend.

- **LocalStack:** A tool that pretends to be AWS, running entirely on your own computer (inside Docker), so you can test real cloud tools (like Terraform) without a real AWS account or any billing risk.

- **moto:** A lighter-weight alternative to LocalStack. Also fakes AWS, but runs as a plain Python program -- **no Docker, no VM needed at all**. This is what we ended up using, because it achieved the same validation goal with far less setup (see Part 3 for why we switched).

- **`moto[server]` / `moto_server`:** The specific mode of moto that starts a small local web server pretending to be AWS, which an external tool like Terraform can be pointed at.

This project's own vocabulary

- **Blast radius:** How far the effects of a change can spread -- i.e., which other resources are connected to (and therefore might be affected by) the thing being changed.

- **Dependency graph:** The map of "what connects to what" that blast radius is calculated from.

- **Risk level:** This tool's verdict (LOW / MEDIUM / HIGH / CRITICAL) on how dangerous a proposed change is.

- **Data Completeness** (originally called "Confidence Score" -- renamed

after a design flaw was found and fixed, see Part 6): a measure of how much of a proposed change's details are fully known before it's applied, not a measure of how "safe" the change is.

- **Rollback plan:** The exact steps to undo a change, computed *before* it's even applied -- not something to improvise later if it goes wrong.

- **Plain-English summary:** A short paragraph explaining a change and its risk in ordinary language, understandable without any cloud background.

Part 3: The Method We Used, and Why Not the Alternatives

The method, named

Pre-deployment static blast-radius analysis of Infrastructure-as-Code.

Broken down: we analyze the infrastructure change **before** ("pre- deployment") it's applied, by reading its description as a file ("static," meaning we don't need to run/watch anything live), tracing how far its effects could spread ("blast-radius analysis"), from Infrastructure-as-Code (a Terraform plan).

Alternative Method A (rejected): Live cloud monitoring

(This was the original plan: AWS CloudTrail + AWS Config + Amazon EventBridge + AWS Lambda + Amazon SNS, watching a live AWS account and reacting to changes as they happen.)

Why we didn't build this: it requires a real, continuously-running AWS account, and one specific piece (AWS Config) has **real, non-zero billing costs** with no generous free tier -- it charges per tracked configuration item and per rule evaluation, from the very first item, regardless of account age. Given an explicit requirement of zero billing risk (a real concern -- a friend of the project owner was once billed unexpectedly on AWS after believing everything was deleted), this method was ruled out. It also only reacts to changes **after** they've already happened, which is structurally the same limitation every existing tool (AWS Config, CloudTrail) already has.

Alternative Method B (rejected): Manual code review only

Just have a senior engineer read every infrastructure change by eye before approving it. This is what most small teams actually do today. **Why it's not enough at scale:** no individual, however senior, has full visibility into every other team's usage of a shared resource in a larger organization -- this is an information-availability problem, not a skill problem. It also doesn't scale to the volume of changes large companies make per day via automated pipelines.

Alternative Method C (considered, partially rejected): Machine learning / AI prediction

Train a model on historical incident data to predict risk. **Why we didn't use this:** there is no large, labeled, real-world dataset of "this exact change caused this exact outage" available to train on. A rule-based system, while less sophisticated-sounding, is fully explainable -- every verdict can be traced to a specific, statable rule, which matters a great deal for a tool that needs to be trusted by engineers. This is a deliberate design choice, not a limitation we're hiding.

Why our chosen method wins for this project's actual goal

- **Zero cost, zero live account needed** -- the entire analysis runs against a saved plan file, matching the "zero billing tolerance" requirement.
- **Catches problems before they happen**, not after -- a stronger

position than any purely reactive tool.

- **Provider-agnostic** -- the exact same code path works for AWS, Azure, and GCP resources in a single plan, because Terraform's plan format is provider-agnostic. Proven for real with a plan tracing a change through Azure and Google Cloud resources in one graph.

- **Fully explainable** -- every risk verdict traces back to a specific, statable rule, unlike a machine-learning prediction.

Part 4: Who / When / Where / Why to Use This Project

Question	Answer
Who	Cloud/DevOps/SRE engineers who manage infrastructure through Terraform; teams at any company past the size where one person can hold the whole system in their head.
When	The moment a change is <i>proposed</i> -- i.e., when a pull request is opened -- not after it's already live.
Where	Plugged into a company's existing CI/CD pipeline (GitHub Actions, or equivalent), alongside their existing Terraform workflow.
Why	Config-change-caused outages are a well-documented, real, recurring category of incident (several famous, public examples exist). This tool is a "shift-left" defense -- catching the problem at proposal time, not at 3am during an outage.

Who it is *not* mainly for: a solo hobbyist with one server and no team -- the core value (catching blind spots across people who don't share full context) mostly appears once more than one person/team shares infrastructure.

Part 5: Complete Step-by-Step Setup From Absolute Zero

Assume: a brand-new Windows laptop, nothing installed, starting completely from scratch.

Step 0 -- Check what you're starting with

Open PowerShell and run:

```
python --version  
git --version
```

If either says "not recognized," you don't have it yet -- install it in the matching step below.

Step 1 -- Install Python

Download and install from <https://python.org> (get the latest 3.x version). **Important:** during installation, check the box that says "Add Python to PATH" -- skipping this causes constant "command not found" errors later.

Verify:

```
python --version
```

Step 2 -- Install Git

Download from <https://git-scm.com>, install with default options.

Verify:

```
git --version
```

Step 3 -- Get the project code

If you have the GitHub link:

```
git clone https://github.com/Shobaa-cloud/HybridCloudImpactAnalyzer_Bot.git  
cd HybridCloudImpactAnalyzer_Bot
```

If you only have a zip file, extract it and open a terminal inside that folder instead.

Step 4 -- Install the Python libraries this project needs

```
pip install -r requirements.txt
```

This reads `requirements.txt` (a list of every library the project needs) and installs each one automatically. **Troubleshooting for this exact step is in Part 6, Error #1.**

Step 5 -- Set up configuration

```
copy .env.example .env
```

Open the new `.env` file in a text editor. For a first run, you don't need to change anything -- it defaults to a zero-setup local database (SQLite).

Step 6 -- Create some example history data

```
python seed_history.py
```

This fills in a handful of example "past incidents" so the dashboard isn't empty on first run. Safe to run more than once -- it skips itself if data already exists.

Step 7 -- Try the command-line version first (simplest, fastest to verify everything works)

```
python cli.py analyze samples/sg_open_ssh.json
```

You should see a full report print to the terminal: risk level, affected resources, recommendations, and a rollback plan. If this works, the core engine is confirmed working.

Step 8 -- Run the full web dashboard

```
python app.py
```

Then open a browser to **`http://127.0.0.1:5000`**. Pick a sample from the dropdown, click "Analyze," and watch the results populate. Press **Ctrl+C** in the terminal to stop it when you're done.

Step 9 -- Run the automated tests (confirms nothing is broken)

```
python -m pytest tests/ -q
```

Every line of dots represents one passing test. If you see **F** instead of a dot, something's broken -- the test's name and error message will tell you exactly what.

Step 10 -- Push this to your own GitHub, so the automated bot can run

1. Create a new, empty repository at <https://github.com/new> (don't initialize it with a README).

2. Point your local project at it and push:

```
git remote set-url origin https://github.com/<your-username>/<your-repo-name>.git
git push -u origin main
```

3. Open a Pull Request that changes a file under `samples/` (edit an existing one, or add a new plan file) -- this triggers the automated check.

4. Watch it run under the "Actions" tab of your repository, and see the bot's comment appear on your pull request within about a minute.

Step 11 (optional) -- Validate against a **real** Terraform-generated plan, not just our sample files

This is the "prove it's not just testing on data we made up" step -- genuinely worthwhile, but optional.

11a. Install Terraform:

```
winget install Hashicorp.Terraform
```

Troubleshooting: see **Part 6, Error #6** (PATH not refreshing after install).

11b. Install moto (a fake-AWS server, no real account needed):

```
pip install "moto[server]"
```

11c. Start the fake-AWS server (leave this running in its own terminal window):

```
python -m moto.server -p 5001
```

11d. Write a small Terraform file describing a fake company setup -- save as `main.tf` in a new empty folder:

```

terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
  access_key = "test"
  secret_key = "test"
  skip_credentials_validation = true
  skip_metadata_api_check = true
  skip_requesting_account_id = true
  s3_use_path_style = true

  endpoints {
    ec2 = "http://localhost:5001"
    iam = "http://localhost:5001"
    sts = "http://localhost:5001"
    s3 = "http://localhost:5001"
  }
}

resource "aws_security_group" "web_sg" {
  name = "web-sg"
  description = "Web server security group"

  ingress {
    from_port = 22
    to_port = 22
    protocol = "tcp"
    cidr_blocks = ["192.168.1.10/32"]
  }

  tags = { Environment = "production" }
}

resource "aws_instance" "web" {
  ami = "ami-12345678"
  instance_type = "t2.micro"
  vpc_security_group_ids = [aws_security_group.web_sg.id]

  tags = { Environment = "production" }
}

```

11e. Run the real Terraform commands:

```

terraform init
terraform plan -out=tfplan
terraform show -json tfplan > real_plan.json

```

11f. Feed the real output through the analyzer:

```

python cli.py analyze real_plan.json

```

If you see a full, correct report (not an error), you've just proven the tool works on genuine Terraform output, not just our own sample files.

Step 12 (optional, advanced) -- Connect a real AWS account, read-only only

Only do this if you specifically want the dashboard to know your account's *current* state (separate from analyzing a proposed plan). This is optional and off by default.

1. In the AWS Console: create an IAM user with **programmatically access only** (no password/console login).
2. Attach the exact policy in `iam-readonly-policy.json` in this project
-- it allows *only* 3 read actions (`DescribeSecurityGroups`, `DescribeInstances`, `DescribeDBInstances`) and nothing else, so even a bug in the code cannot create or change anything.
3. Add the access key to `.env`.
4. **Also set up a \$0 AWS Budget alert** (AWS Console -> Billing -> Budgets) -- free to create, emails you instantly if any charge ever appears, independent of anything in this project.
5. Run:

```
python sync_current_state.py
```

Part 6: Troubleshooting Log -- Every Real Error We Hit, and Why

These are not hypothetical -- every single one of these actually happened while building this project the first time.

Error #1 -- `pip install` shows a wall of `WARNING: The script ... is installed in '...\Scripts' which is not on PATH`

Why it happens: `pip` installs some tools' command-line shortcuts into a folder that isn't automatically on your terminal's search path. **Fix:** Harmless -- completely safe to ignore for this project. Nothing we use depends on those specific shortcuts being directly runnable.

Error #2 -- `python -m pytest` says `No module named pytest`, even though you just installed it

Why it happens: your computer can have more than one Python installation, and `pip install` and `python` might silently be pointing at two different ones. **Fix:** Run `python -c "import sys; print(sys.executable)"` to see exactly which Python is active, then reinstall using that same exact path: `<that path> -m pip install pytest`.

Error #3 -- `git push` fails with `Permission ... denied to <username> (HTTP 403)`

Why it happens: the GitHub account saved in your computer's login doesn't have write access to that specific repository (e.g., it belongs to a teammate, and you were never added as a collaborator). **Fix:** Either ask the repository's owner to add your account as a collaborator with write access, or push to your **own** new, empty repository instead (`git remote set-url origin <your-own-repo-url>`).

Error #4 -- The bot's report crashes with `UnicodeEncodeError: 'charmap' codec can't encode character '\U0001f6e1'`

Why it happens: Windows terminals sometimes default to an old text encoding that can't display emoji characters (like 🐼), which our report uses for readability. **Fix:** Add this near the top of the script printing the report:

```
import sys
try:
    sys.stdout.reconfigure(encoding="utf-8")
except AttributeError:
    pass
```

This doesn't affect GitHub Actions at all (Linux there is UTF-8 by default) -- it's purely a local-Windows-terminal quirk.

Error #5 -- `wsl --install` fails with `Forbidden (403)`.

Why it happens: this command tries to download a Linux kernel update from Microsoft's servers, and a 403 almost always means **something on your network is blocking that download** -- a restrictive Wi-Fi (campus/ hostel networks are common culprits), a firewall, or antivirus software. **Fix:** Try a different network (e.g., a mobile hotspot) and re-run the command. If that's not possible, there's a manual installation path that avoids the same downloader (enable the Windows features via `dism.exe` directly, then download the kernel update `.msi` through a normal browser instead of the command line).

Error #6 -- `terraform: command not found`, right after `winget install Hashicorp.Terraform` said it succeeded

Why it happens: `winget` adds the new program's location to your system's PATH setting, but your **already-open** terminal window loaded its PATH before that change happened, so it doesn't know about it yet. **Fix:** Close and reopen your terminal (simplest), or find the exact install location and call the program using its full path as a one-time workaround:

```
Get-ChildItem -Path "$env:LOCALAPPDATA\Microsoft\WinGet" -Filter "terraform.exe"
-Recurse
```

Error #7 -- Can't copy-paste between a VirtualBox virtual machine and the main computer

Why it happens: VirtualBox does not share the clipboard between the virtual machine and the host computer by default -- this requires an extra helper package ("Guest Additions") to be installed *inside* the virtual machine, plus a setting enabled in the VirtualBox window itself. **Fix (inside the Linux VM):**

```
sudo apt install -y virtualbox-guest-utils virtualbox-guest-x11
sudo reboot
```

Then in the VirtualBox window's menu (not inside Linux): **Devices -> Shared Clipboard -> Bidirectional**.

Error #8 -- Docker Desktop requires WSL2, which requires a restart, which requires admin rights

Why it happens: Docker Desktop on Windows fundamentally depends on a lightweight virtual Linux environment (WSL2) to function -- this isn't a bug, it's how Docker Desktop for Windows is architected. Enabling this Windows feature always requires Administrator rights and a restart to take effect. **Fix:** Either run the setup commands from an Administrator PowerShell window and restart when asked, or -- what we actually ended up doing -- **skip Docker entirely** and use `moto` instead (a pure-Python fake-AWS server, explained in Part 2, that needed none of this).

Error #9 -- The "confidence score" was quietly measuring the wrong thing

Not a crash -- a design bug, caught by asking "why would history make something more confident, when the environment could have completely changed since then?" The original formula gave extra points just because a similar change had been analyzed before, regardless of whether that past analysis was actually accurate. **Fix:** removed that boost entirely; the score now measures only how complete the *current* change's data is (nothing about the past), and was relabeled "Data Completeness" everywhere in the

UI/CLI/reports so it stops implying "this means it's safe."

Error #10 -- A change inside a Terraform module reported 0 affected resources, even though a real dependent resource existed

Why it happens: the code that builds the "what depends on what" map only ever looked at `configuration.root_module.resources` in Terraform's plan file -- but resources declared inside a **module** (`module "app" { ... }`) live in a different part of that file (`configuration.root_module.module_calls[...].module.resources`), which was never being read. Since most real companies organize their Terraform into modules, this was a significant real gap, only discovered because we tested against a real Terraform-generated file (see Step 11) instead of only our own hand-made samples. **Fix:** rewrote the dependency-parsing function to recursively walk into every nested module, correctly prefixing addresses the same way real Terraform does (`module.app.aws_instance.web`).

Part 7: Honest Limitations -- What's Solid, What Isn't

Solid, tested, defensible:

- The core parsing -> graph -> blast-radius -> risk -> recommendation -> rollback pipeline. Tested against both hand-written samples and one genuinely real Terraform-generated plan.
- Multi-cloud tracing (AWS + Azure + GCP in one dependency graph), proven with a real cross-provider example.
- The CI bot, including its ability to actually block a merge, not just comment.

Real, honest gaps, if this were handed to an actual company today:

- The GitHub Action's trigger (watching for hand-edited sample files) is a demo convenience -- a real company would need an actual `terraform plan` step wired into their pipeline instead.
- The bot's "similar past changes" history does not currently persist between separate CI runs (each run gets a brand-new, empty database) -- it would need a real, persistent database wired in via a secret to be useful there.
- No multi-user accounts or permission boundaries -- this is a single-team/local tool as built, by design, not accidentally.
- The optional live-AWS features (`aws_enrichment.py`, `live_state.py`) are correct by careful inspection and IAM-policy scoping, but have not been run against a real AWS account in this build.

Part 8: Cheat Sheet for Explaining This Project Out Loud

- **"What does it do?"** -- Reads a proposed infrastructure change before it's applied, and predicts what else it will affect and how risky it is, instead of finding out after something breaks.
- **"How is it different from existing tools?"** -- Existing tools (AWS Config, CloudTrail) tell you what changed *after* it happened. This works *before*, on the proposed plan.
- **"Is it AI?"** -- No. It's deterministic, rule-based automation -- every verdict traces back to a specific, storable rule, which is a deliberate choice for explainability, not a limitation.
- **"Does it watch my AWS account 24/7?"** -- No. It only runs the moment someone manually asks it to, or when a pull request is opened.
- **"What's the biggest proven bug you found and fixed?"** -- The Terraform-module dependency gap (Part 6, Error #10) -- found by testing against a real, tool-generated plan instead of only our own samples.
- **"What would you do with more time?"** -- Wire a real `terraform plan` step into CI instead of the demo file-watching trigger, add a persistent database for the bot's history, and get it running against an actual company's (redacted) Terraform code.